

Department of Computer Science and Engineering
University of Dhaka

CSE 4113: Internet Programming Lab

CSE SmartPortal

A Unified Academic Resource Management System

Lab 4: Software Design Document (SDD)

Version 1.0 — April 14, 2026

Team Name: LogicLoop

Abdullah Al Arman Emon	Roll: 50
Shah Jamal Islam	Roll: 81
Suhail Tanvir Nahin	Roll: 82
Md. Rakib Hossain	Roll: 88
Imran Shorif Shuvo	

Submitted To

Dr. Mamun Or Rashid, Professor, CSEDU
Md. Fahim Arefin, Lecturer, CSEDU
Md. Ahasanul Alam, Lecturer, CSEDU

Related SRS Version: v2.0

Repository URL: github.com/team-logicloop/cse-smartportal

Revision History

Version	Date	Author	Description
1.0	April 14, 2026	Team LogicLoop	Initial SDD, aligned with SRS v2.0 and implemented codebase; covers all 15 functional modules.

Contents

Revision History	1
1 Team and Project Information	5
2 System Overview	6
2.1 Problem Summary	6
2.2 Target Users	6
2.3 Project Goals	6
2.4 Out of Scope	7
3 Architecture Design	8
3.1 High-Level Architecture	8
3.1.1 Main Components / Services	8
3.1.2 Data Flow Summary	8
3.1.3 External Integrations	9
3.2 Component Breakdown	9
3.3 Key Design Decisions	10
4 Data Design	12
4.1 Core Entities	12
4.2 Key Relationships	13
4.3 Constraints	14
4.4 Data Lifecycle Notes	14
5 Core Flow Design	16
5.1 Flow 1 – Student Course Enrolment and Assignment Submission	16
5.2 Flow 2 – Admin Finance Event Creation and Student Payment Verification	17
5.3 Flow 3 – Multi-Step Admission Application Submission	18
6 Security and Access Design	20
6.1 Authentication Approach	20
6.2 Authorisation Model	21
6.3 Sensitive Data Handling	21
6.4 Audit and Logging Plan	22
7 Implementation Plan	23
7.1 Sprint 1 – Ongoing Features	23

7.2	Technical Risks	24
7.3	Mitigation Plan	25
8	Testing Plan (Design Level)	26
8.1	Unit Testing Targets	26
8.2	Integration Testing Targets	27
8.3	End-to-End (E2E) Testing Targets	27
8.4	Known Hard-to-Test Areas	28
9	Exit Checklist	29
A	Requirements Traceability Matrix	30
B	API Route Summary	32
C	Frontend Route Map	33

List of Tables

3.1	Component Breakdown	9
3.2	Key Design Decisions	11
4.1	Core Database Entities	12
A.1	SRS to SDD Traceability	30
B.1	Implemented API Route Summary	32
C.1	Frontend Route Map	33

1. Team and Project Information

Field	Detail
Team Name	LogicLoop
Project Name	CSE SmartPortal – Unified Academic Resource Management System
Repository URL	github.com/team-logicloop/cse-smartportal
Commit / Tag	v2.0-srs-aligned
Date	April 14, 2026
Related SRS Version	v2.0 (Updated, peer-review aligned)

Team Members and Responsibilities:

Name	Roll	Primary Responsibility
Abdullah Al Arman Emon	50	Backend architecture, authentication, course management APIs
Shah Jamal Islam	81	Frontend React components, routing, student dashboard
Suhail Tanvir Nahin	82	Finance, equipment borrowing, admin modules
Md. Rakib Hossain	88	Database design, teacher profile, resource hub
Imran Shorif Shuvo	–	Admission hub, exam scheduling, meetings, testing

2. System Overview

2.1 Problem Summary

The Department of Computer Science and Engineering (CSEDU) at the University of Dhaka has historically managed academic operations through fragmented, manual processes: notice distribution via printed boards, ad-hoc email chains for meeting invitations, spreadsheets for equipment inventory, and face-to-face submission of course assignments. These processes are error-prone, difficult to audit, and inaccessible to students and faculty outside the department premises.

CSE SmartPortal addresses this gap by providing a single, role-based web application that centralises all departmental academic operations, digital content management, and administrative workflows into one coherent platform.

2.2 Target Users

Actor	Auth Required	Primary Activities
Visitor	No	Browse public website, read notices, submit admission application
Student	Yes	Access courses, submit assignments, view exams, borrow equipment, pay fees
Teacher	Yes	Manage courses, post assignments, grade submissions, edit profile, RSVP meetings
Admin	Yes	Full system control: manage content, users, finance, exams, admissions, equipment

2.3 Project Goals

- G1:** Replace all manual academic workflows with a unified digital platform.
- G2:** Provide a secure, JWT-authenticated, role-based access control system.
- G3:** Enable teachers to create and manage courses with assignment and resource delivery.
- G4:** Offer students a self-service portal for enrolment, submissions, finance, and equipment.
- G5:** Give administrators a powerful dashboard for all content and operational management.

G6: Expose a public-facing departmental website for visitors and prospective students.

G7: Ensure the system is maintainable, scalable, and deployable to production.

2.4 Out of Scope

- Live video or virtual classroom integration (e.g., embedded Zoom/Meet).
- Automated grading or plagiarism detection.
- Integration with the university-wide ERP or student information system.
- Mobile native applications (iOS/Android); the portal is responsive web only.
- Email or SMS notification delivery (only in-portal notifications are in scope).
- Real payment gateway processing (a mock gateway is used for this version).

3. Architecture Design

3.1 High-Level Architecture

CSE SmartPortal follows a **three-tier client-server architecture** with a clear separation between the presentation tier (React/Vite SPA), the application/logic tier (FastAPI REST API), and the data tier (relational database via SQLAlchemy ORM).

Architectural Pattern

Pattern: Three-Tier MVC-inspired REST Architecture

Frontend–Backend communication: JSON over HTTP/HTTPS (REST API)

Authentication: Stateless JWT bearer tokens

File serving: Static file mount at `/resources` (local filesystem)

3.1.1 Main Components / Services

1. **React SPA (Frontend)** – Single-page application built with React 18 + Vite, styled with Tailwind CSS, with React Router v6 for client-side routing and role-based route guards.
2. **FastAPI Backend** – Python 3.12 REST API server. Routers are modularly organised by feature domain under `app/Sami` and `app/Faiak` sub-packages.
3. **SQLAlchemy ORM + Database** – Declarative models with Alembic migrations. SQLite is used in development; the system is PostgreSQL-ready via environment variable swap.
4. **Static File Server** – Files uploaded through the resource hub or profile images are stored on the local filesystem and served via FastAPI’s static mount.
5. **JWT Auth Service** – Implemented as a dependency injector within FastAPI using `python-jose` and `bcrypt` for token issuance and password hashing.

3.1.2 Data Flow Summary

1. The user’s browser loads the React SPA from the Vite dev/production server.
2. The SPA makes authenticated HTTPS requests to the FastAPI backend carrying a JWT Bearer token in the `Authorization` header.

3. FastAPI validates the token, checks the user's role via dependency injection, and routes the request to the appropriate router.
4. The router interacts with the database through SQLAlchemy session objects, performing ORM queries, and returns JSON responses.
5. File uploads are saved to the `/resources` directory and the stored path is returned in the JSON response; files are later served via the static mount.

3.1.3 External Integrations

Integration	Type	Purpose
Mock Payment Gateway	Internal component	Simulates fee payment; generates a transaction ID for students
People Directory JSON	Static data file	Supplies faculty listing for the public People Directory page
Meeting URL (Google Meet / Zoom)	External link	Meetings store external conference URLs; no API integration

3.2 Component Breakdown

Table 3.1: Component Breakdown

Component	Responsibility	Inputs	Outputs
React SPA	Renders UI, manages routing & state, sends API requests	User interactions, JWT token	HTTP requests, rendered views
Vite build			
FastAPI Auth Router	Register, login, password change; issue JWT	Email, password, role	JWT token, user record
Course Router	CRUD courses, enrolment, archival	Course data, passcode	Course objects, enrolments
Assignment Router	Post assignments, receive submissions, grading	Assignment details, files	Assignment/submission records
Student & teacher sub-routes			
Notice Router	Create/delete notices with batch scope	Notice payload, attachments	Notice records, filtered lists
Admin & student routes			
Event Router	Manage departmental events	Event data, media URLs	Event records

(Table 3.1 continued)

Component Notes	Responsibility	Inputs	Outputs
Public list endpoint Exam Router	CRUD exam schedule, filterable list	Exam details, batch, type	Exam records
Admin creates, stu- dents view Admission Router	Multi-step form submis- sion & admin review	Form data, docu- ment files	Admission records
Guest & admin routes Meeting Router	Schedule meetings, archive, RSVP	Meeting data, RSVP response	Meeting/RSVP records
/v1/meetings Equipment Router	Inventory management, borrow requests	Equipment data, borrow request	Equipment records
Admin & student routes Finance Router	Fee events, mock pay- ments, verification	Finance event, transaction ID	Payment records
/v1/finance Teacher Profile Router	Profile CRUD, research papers, links	Profile fields, pa- per URLs	Teacher profile record
/v1/teacher/profile Student Dashboard Router	Aggregated summary for student view	JWT token	Summary JSON
/student/dashboard SQLAlchemy ORM	Abstract DB interaction into Python objects	ORM model in- stances	DB rows, query result sets
Alembic migrations Static File Mount	Serve uploaded files (PDFs, images)	HTTP GET re- quest	Binary file response
Mounted at /resources JWT Dependency	Validate bearer token, extract user & role	HTTP Authoriza- tion header	User object or HTTP 401/403
Reused as FastAPI dep. RequireAuth Com- ponent React component	Frontend route guard by role	Current route, user role	Render or redirect to /login

3.3 Key Design Decisions

Table 3.2: Key Design Decisions

Decision	Options	Chosen Option	Reason
Backend framework	Django REST Framework, Flask, FastAPI	FastAPI (Python 3.12)	Automatic docs, async support, type-safe Pydantic schemas, fast dev velocity
Authentication	Session cookies, OAuth2 SSO, JWT	JWT Bearer tokens	Stateless; supports horizontal scaling; no server-side session store required
Frontend framework	Next.js, Vue 3, React 18	React 18 + Vite	Largest ecosystem, team familiarity, fast HMR with Vite
Database (dev)	MySQL, PostgreSQL, SQLite	SQLite (dev) / PostgreSQL (prod)	Zero-config local dev; single env-var swap to PostgreSQL for production
ORM	Raw SQL, SQLAlchemy, Tortoise ORM	SQLAlchemy declarative	Mature, well-documented, integrates with Alembic for migrations
File storage	Cloud (S3), database BLOB, local filesystem	Local filesystem (/resources)	Minimal external dependencies for lab scope; swappable to S3 in production
API versioning	No versioning, URL path prefix, header	URL path prefix (/v1/)	Simple, visible, easy to route; supports future v2 without breaking clients
Role enforcement	Middleware, endpoint decorator, dependency	FastAPI dependency injection	Composable, reusable; allows per-endpoint role checks without boilerplate
Course enrolment	Manual approval, invite email, passcode	Auto-generated 10-char passcode	Simple, offline-friendly; teacher shares code verbally or via notice
CSS framework	Vanilla CSS, Bootstrap, Tailwind CSS	Tailwind CSS	Utility-first; avoids CSS specificity wars; consistent responsive design

4. Data Design

4.1 Core Entities

Table 4.1: Core Database Entities

Entity	Purpose & Key Attributes
users	Core authentication record. Fields: <code>id</code> (PK), <code>email</code> (unique), <code>hashed_password</code> , <code>role</code> (enum: <code>student/teacher/admin</code>), <code>created_at</code> .
students	Student profile linked 1:1 to <code>users</code> . Fields: <code>id</code> , <code>user_id</code> (FK), <code>batch</code> , <code>bio</code> , <code>phone</code> , <code>profile_image</code> .
teachers	Teacher profile linked 1:1 to <code>users</code> . Fields: <code>id</code> , <code>user_id</code> (FK), <code>first_name</code> , <code>last_name</code> , <code>title</code> , <code>department</code> , <code>bio</code> , <code>phone</code> , <code>profile_image</code> , social link fields.
courses	Course entity. Fields: <code>id</code> , <code>title</code> , <code>description</code> , <code>semester</code> , <code>batch</code> , <code>type</code> (Theory/Lab), <code>code</code> (unique, 10-char passcode), <code>image_url</code> , <code>other_links</code> , <code>running</code> (bool), <code>teacher_id</code> (FK).
student_courses	M:N junction for enrolment. Fields: <code>student_id</code> (FK), <code>course_id</code> (FK). Composite PK.
schedules	Weekly class slots. Fields: <code>id</code> , <code>course_id</code> (FK), <code>teacher_id</code> (FK), <code>day</code> , <code>start_time</code> , <code>end_time</code> , <code>room</code> .
assignments	Homework or resource attached to a course. Fields: <code>id</code> , <code>course_id</code> (FK), <code>title</code> , <code>description</code> , <code>type</code> (Homework/Resource), <code>max_marks</code> , <code>given_date</code> , <code>due_date</code> , <code>file_links</code> (JSON array).
submissions	Student submission for an assignment. Fields: <code>id</code> , <code>assignment_id</code> (FK), <code>student_id</code> (FK), <code>file_links</code> (JSON), <code>submitted_at</code> , <code>marks</code> , <code>checked</code> (bool).
notices	Department notice. Fields: <code>id</code> , <code>title</code> , <code>subtitle</code> , <code>body</code> , <code>source</code> (enum), <code>batch</code> (nullable), <code>attachments</code> (JSON), <code>created_at</code> .
events	Departmental events. Fields: <code>id</code> , <code>name</code> , <code>description</code> , <code>start_date</code> , <code>end_date</code> , <code>location</code> , <code>registration_deadline</code> , <code>image_url</code> , <code>video_url</code> , <code>registration_link</code> .

(Table 4.1 continued)

Entity	Purpose & Key Attributes
exams	Exam schedule records. Fields: <code>id</code> , <code>subject</code> , <code>date_time</code> , <code>duration_minutes</code> , <code>batch</code> , <code>room</code> , <code>type</code> (Midterm/Final).
meetings	Departmental meeting. Fields: <code>id</code> , <code>title</code> , <code>date_time</code> , <code>url</code> , <code>is_archived</code> (bool).
rsvps	Meeting RSVP. Fields: <code>id</code> , <code>meeting_id</code> (FK), <code>user_id</code> (FK), <code>response</code> (Yes/No). Unique per (meeting, user).
research_papers	Teacher research paper links. Fields: <code>id</code> , <code>teacher_id</code> (FK), <code>title</code> , <code>url</code> .
equipments	Lab equipment inventory. Fields: <code>id</code> , <code>name</code> , <code>description</code> , <code>quantity_available</code> , <code>image_url</code> .
student_equipments	Borrow transaction. Fields: <code>id</code> , <code>student_id</code> (FK), <code>equipment_id</code> (FK), <code>quantity</code> , <code>start_date</code> , <code>end_date</code> , <code>returned</code> (bool).
finance_events	Fee payment event. Fields: <code>id</code> , <code>title</code> , <code>amount</code> , <code>batch</code> , <code>deadline</code> .
student_payments	Payment record. Fields: <code>id</code> , <code>student_id</code> (FK), <code>finance_event_id</code> (FK), <code>transaction_id</code> , <code>status</code> (pending/paid), <code>created_at</code> , <code>verified_at</code> .
admission_form	Multi-step admission application. Fields: <code>id</code> , <code>email</code> , personal info fields, academic background fields, document paths, <code>form_given_on</code> .
activity_logs	Teacher action audit log. Fields: <code>id</code> , <code>teacher_id</code> (FK), <code>message</code> , <code>timestamp</code> .
missing_classes_on_monthly	Monthly attendance gap. Fields: <code>id</code> , <code>student_id</code> (FK), <code>month</code> , <code>year</code> , <code>percentage</code> . Unique per (student, month, year).

4.2 Key Relationships

- `users` → 1:1 → `students` (on role = student)
- `users` → 1:1 → `teachers` (on role = teacher)
- `students` → M:N → `courses` via `student_courses`
- `teachers` → 1:M → `courses`, `schedules`, `research_papers`
- `courses` → 1:M → `schedules`, `assignments`
- `assignments` → 1:M → `submissions`
- `students` → 1:M → `submissions`, `student_equipments`, `student_payments`, `missing_classes_on_monthly`
- `meetings` → 1:M → `rsvps`
- `finance_events` → 1:M → `student_payments`
- `equipments` → 1:M → `student_equipments`

4.3 Constraints

Entity / Field	Constraint	Type
users.email	Must be unique across all users	UNIQUE
users.role	Must be one of: student, teacher, admin	ENUM
courses.code	Auto-generated 10-char alphanumeric; unique	UNIQUE
student_courses	Composite PK on (student_id, course_id)	PK
rsvps	Unique per (meeting_id, user_id); upsert on resubmit	UNIQUE
missing_classes_on_month	Unique per (student_id, month, year)	UNIQUE
assignments.type	Must be Homework or Resource	ENUM
exams.type	Must be Midterm or Final	ENUM
student_payments.status	Must be pending or paid; default: pending	ENUM/DEFAULT
admission_form.form_given_on	Server-side timestamp; NOT NULL	NOT NULL
teachers.email	Must end with @cse.du.ac.bd	VALIDATED (app layer)
students.email	Must end with @cs.du.ac.bd	VALIDATED (app layer)
submissions.checked	Default false; set true on grading	DEFAULT
meetings.is_archived	Default false; set true on archival	DEFAULT
courses.running	Default true; set false on archival	DEFAULT

4.4 Data Lifecycle Notes

- **Passwords:** Plaintext passwords are hashed with bcrypt immediately upon registration; the plaintext value is never persisted.
- **JWT tokens:** Tokens are stateless and expire after 24 hours (configurable). No server-side token store is maintained.
- **File uploads:** Uploaded files (assignment submissions, profile images, admission documents, resources) are stored on the local **/resources** filesystem. Deletion of a DB record does not automatically purge the associated file.
- **Course archival:** Archiving a course sets **running = false**; the record and all associated assignments/submissions are retained for historical reference.

- **Meeting archival:** Archiving sets `is_archived = true`; the record is hidden from the public list but not deleted.
- **Equipment return:** On marking equipment returned, `returned = true` and `quantity_available` on the equipment record is incremented.
- **Payment verification:** Admin verification sets `status = "paid"` and records `verified_at` timestamp; the payment record is retained permanently.
- **Database migrations:** All schema changes are managed via SQLAlchemy declarative models; `Base.metadata.create_all` bootstraps tables on first run.

5. Core Flow Design

5.1 Flow 1 – Student Course Enrolment and Assignment Submission

Flow 1 Overview

This flow covers the end-to-end journey of a student joining a course using a passcode and subsequently submitting a homework assignment.

Trigger: A student receives a 10-character enrolment passcode from their teacher and navigates to `/enroll-course`.

Steps:

1. **Authentication check:** `RequireAuth` component verifies a valid JWT with role `student`. If absent, redirect to `/login`.
2. **Passcode submission:** Student enters the passcode and submits. SPA sends `POST /v1/courses/enrol` with `{code: "XXXXXXXXXX"}`.
3. **Backend validation:** FastAPI queries `courses` for matching `code`. If not found, returns HTTP 404.
4. **Enrolment record creation:** A row is inserted into `student_courses` linking the student's ID and the course ID. Returns HTTP 200.
5. **Course listing:** Student navigates to `/my-courses`; SPA calls `GET /v1/courses/my-courses`, which returns all running courses for the student.
6. **Assignment viewing:** Student opens a course classroom; SPA fetches assignments from the course's assignment endpoint. Assignments of type `Homework` with a future `due_date` are displayed.
7. **File upload:** Student selects a file and clicks Submit. SPA sends `POST /student/assignment/su` with a multipart form containing the file and `assignment_id`.
8. **Submission record creation:** Backend saves the file to `/resources`, inserts a row in `submissions` with `submitted_at = now()`, and returns HTTP 201.
9. **Confirmation:** SPA displays a success toast; the assignment card shows a "Submitted" badge.

Success Outcome: A `submissions` record exists in the database linked to the student and assignment; the uploaded file is accessible at `/resources/{filename}`.

Failure / Edge Cases:

- Invalid passcode → HTTP 404, error toast displayed.
- Student already enrolled in course → HTTP 400 (duplicate key), error toast.
- Submission after `due_date` → Backend rejects with HTTP 400; frontend shows deadline-passed warning.
- File exceeds 10 MB → Backend returns HTTP 413; frontend shows file size error.

Sequence Summary:

Student Browser → POST `/v1/courses/enrol` → FastAPI `CourseRouter` → DB: `SELECT courses WHERE code=?` → DB: `INSERT student_courses` → HTTP 200 response → SPA updates course list

Student Browser → POST `/student/assignment/submit` (multipart) → FastAPI `AssignmentRouter` → File saved to `/resources` → DB: `INSERT submissions` → HTTP 201 response → SPA shows success badge

5.2 Flow 2 – Admin Finance Event Creation and Student Payment Verification

Flow 2 Overview

This flow documents the lifecycle of a departmental fee collection: admin creates a finance event, the student submits a mock payment, and the admin verifies it.

Trigger: Admin logs in and navigates to `/admin-finance` to create a new fee event for a specific batch.

Steps:

1. **Authentication check:** `RequireAuth` verifies JWT role = `admin`; redirects if unauthorised.
2. **Finance event creation:** Admin fills title, amount, batch, and deadline fields. SPA sends POST `/v1/finance/events`.
3. **Backend creation:** FastAPI validates the payload (Pydantic schema); inserts a `finance_events` record. Returns HTTP 200 with the new event object.
4. **Student views event:** Student navigates to `/finance`; SPA calls GET `/student/finance/event`; the new event appears in the list.
5. **Mock payment:** Student clicks “Pay”; the `MockPaymentGateway` component renders, simulates a payment, and produces a transaction ID string.
6. **Payment submission:** Student submits the transaction ID. SPA sends POST `/v1/finance/payments` with `{finance_event_id, transaction_id}`.

7. **Payment record creation:** Backend inserts a `student_payments` row with `status = "pending"` and `created_at = now()`. Returns HTTP 200.
8. **Admin reviews payments:** Admin sees the pending payment in `/admin-finance`; SPA lists all payment records.
9. **Verification:** Admin clicks “Verify”. SPA sends `PUT /v1/finance/verify/{payment_id}`.
10. **Status update:** Backend sets `status = "paid"` and `verified_at = now()`. Returns HTTP 200.
11. **Confirmation:** Student’s finance page now shows the payment as “Paid”.

Success Outcome: `student_payments.status = "paid"` with a non-null `verified_at` timestamp; both admin and student views reflect the verified state.

Failure / Edge Cases:

- Missing required fields in finance event creation → HTTP 422 (Pydantic validation error).
- Student submits duplicate payment for same event → HTTP 400 conflict.
- Admin attempts to verify a non-existent payment ID → HTTP 404.
- Student’s batch does not match the event’s batch scope → Event not returned in student’s list.

5.3 Flow 3 – Multi-Step Admission Application Submission

Flow 3 Overview

A prospective student (unauthenticated visitor) completes the multi-step admission form including document uploads; admin then reviews the application.

Trigger: A visitor navigates to `/apply` on the public site.

Steps:

1. **Multi-step wizard displayed:** The `ApplicationForm` component renders with a progress indicator showing three steps.
2. **Step 1 – Personal Information:** Visitor enters name, email, date of birth, program, and uploads a profile image. Frontend validates required fields inline.
3. **Step 2 – Academic Background:** Visitor enters qualification, institution, field of study, graduation date, and GPA.
4. **Step 3 – Document Uploads:** Visitor uploads required document, academic transcript, recommendation letter, and personal essay (PDF/image).
5. **Final submission:** SPA compiles all data into a multipart form and sends `POST /guest/admission/apply`.

6. **Backend processing:** FastAPI saves uploaded files to `/resources`, records file paths in the `admission_form` entity, sets `form_given_on = now()`, and returns HTTP 201.
7. **Success confirmation:** SPA shows a confirmation message with an application reference.
8. **Admin review:** Admin navigates to `/admin-dashboard`, calls `GET /admin/admission/list` (sorted newest first), and views or deletes records.

Success Outcome: `admission_form` record exists with all document paths stored and a non-null `form_given_on`.

Failure / Edge Cases:

- Missing required documents → Frontend inline validation blocks progression to Step 3.
- File type not accepted (e.g., executable) → Backend returns HTTP 415.
- Network failure mid-upload → Form state is preserved in React state; user can retry from current step.

6. Security and Access Design

6.1 Authentication Approach

The system uses **JSON Web Token (JWT)** authentication implemented with the `python-jose` library on the FastAPI backend.

- On successful login (`POST /v1/auth/login`), the server generates a signed JWT containing the user's `id`, `email`, and `role`.
- Tokens expire after **24 hours** (configurable via environment variable).
- The SPA stores the token in memory/local storage and attaches it as a **Authorization: Bearer {token}** header on every protected API request.
- Expired or tampered tokens return HTTP 401; the frontend `RequireAuth` component redirects to `/login`.
- Passwords are hashed using **bcrypt** before storage. Plaintext passwords are never written to the database or logs.

6.2 Authorisation Model

Role	Permitted API Prefixes and Actions
Visitor (unauthenticated)	Public endpoints only: /, /people, /chairman, /notice-board, /meetings, /studentEventShow, /admission-hub, /apply (guest admission), GET /admin/events/list.
Student	All visitor routes plus: /student/* (dashboard, notice, exam, equipment, assignment, classes, settings, finance). Can enrol in courses (POST /v1/courses/enrol), submit assignments, borrow equipment, submit payments, RSVP meetings.
Teacher	All student read routes plus: /v1/courses/* (create, archive), /v1/teacher/* (dashboard, profile), /teacher/classes/* (assignment management, grading, view submissions), RSVP meetings. Cannot access admin routes.
Admin	Full access to all routes including /admin/*: notice, event, exam, admission, equipment, finance verification, attendance recording, user management. Also inherits all teacher and student read permissions.

Enforcement layers:

1. **Backend (primary):** Every protected FastAPI endpoint declares a dependency on `get_current_user(role="...")`, which validates the JWT and checks the role. Unauthorised access returns HTTP 403.
2. **Frontend (secondary / UX):** The `RequireAuth` React component checks the stored JWT role before rendering a route. This is a UX convenience layer only; the backend is the authoritative enforcement point.

6.3 Sensitive Data Handling

Data Type	Handling Policy
Passwords	Hashed with bcrypt (cost factor 12) before DB insertion; never logged or returned in API responses.
JWT Secret Key	Stored in <code>SECRET_KEY</code> environment variable; never hard-coded in source.
Database URL	Stored in <code>DATABASE_URL</code> environment variable; not committed to Git.
Uploaded documents	Stored on the server filesystem; accessible only via the <code>/resources</code> static mount. In production, access should be restricted by authentication middleware.
Transaction IDs	Stored as plain strings; no real financial data is processed (mock gateway only).
Email addresses	Used as unique identifiers; role-constrained by domain (<code>@cs.du.ac.bd</code> for students, <code>@cse.du.ac.bd</code> for teachers).

6.4 Audit and Logging Plan

- **Teacher activity logs:** The `activity_logs` table records a timestamped message for each significant teacher action (course creation, assignment posting). Queryable by admin via the admin dashboard.
- **Payment audit trail:** `student_payments` records both `created_at` and `verified_at` timestamps, providing a complete audit trail of fee processing.
- **Admission submission timestamps:** `admission_form.form_given_on` records the exact UTC datetime of each application.
- **Server-level logging:** FastAPI's default Uvicorn access logs capture all HTTP requests, response codes, and timestamps for operational monitoring.
- **Error responses:** All 4xx and 5xx errors return structured JSON `{"detail": "...}"` bodies; raw stack traces are never exposed to clients.

7. Implementation Plan

7.1 Sprint 1 – Ongoing Features

The following modules have been fully implemented and are aligned with SRS v2.0:

Module	Implemented Features
Authentication	Register, login (JWT), password change, RBAC
Public Website	Home, People Directory, Chairman, Notice Board, Events (public), Meetings (public), Admission Hub pages
Notice Management	Admin create/delete; student batch-filtered list; file attachment support
Event Management	Admin CRUD; public event list; media URL support
Course Management	Teacher create; student enrol via passcode; active/archived lists; archival
Assignment & Submissions	Teacher post; student submit with file upload; teacher view/grade
Resource Hub	Static file serving; student download list
Exam Scheduling	Admin CRUD exams; filterable list (batch, type, room); student view
Admission Hub	Multi-step form; document uploads; admin list/delete
Departmental Meetings	Admin schedule/archive; RSVP (Yes/No); public list
Equipment Borrowing	Admin inventory CRUD; student borrow request; return tracking
Finance Management	Admin fee events CRUD; student mock payment; admin verification; mock gateway component
Teacher Profile	Profile CRUD; research papers; social/academic links; dashboard summary
Student Dashboard	Summary: courses, exams, notices, finance obligations
Attendance Tracking	Admin insert monthly missing-class %; student view
Admin Dashboard	Navigation hub to all management panels; teacher activity logs

Feature	Description	Priority
Production deployment	Docker containerisation; Nginx reverse proxy; PostgreSQL production DB; HTTPS via Let's Encrypt	High
Resource access control	Restrict <code>/resources</code> static files to authenticated users; prevent direct URL access by unauthenticated visitors	High
Email notifications	Send email on notice publish, exam schedule, payment verification	Medium
Search and filtering	Global search across courses, notices, events on dashboard	Medium
Pagination	Paginate long lists (notices, admissions, payments, submissions) to support scale	Medium
In-portal notifications	Real-time or polling-based notification bell for new notices, assignments, meeting invitations	Low
Equipment image upload	Allow admin to upload actual images for equipment inventory (currently URL-based)	Low

7.2 Technical Risks

Risk	Description	Likelihood
Local file storage limitations	Files in <code>/resources</code> are not replicated; a server failure causes data loss	High
SQLite concurrency	SQLite has write-lock limitations under concurrent access; unsuitable for production load	High
CORS misconfiguration	Wildcard CORS (*) in development must be restricted before production deployment	Medium
JWT secret exposure	If <code>SECRET_KEY</code> is committed to Git, all tokens can be forged	Medium
File type validation gap	No server-side MIME sniffing; malicious files could be uploaded	Medium
Stale frontend auth state	Expired JWT in localStorage causes silent 401 errors if frontend does not handle token refresh	Low

7.3 Mitigation Plan

Risk	Mitigation
Local file storage	Migrate to S3-compatible object storage (e.g., AWS S3 or MinIO) in Sprint 2
SQLite concurrency	Switch DATABASE_URL to PostgreSQL before production; already supported via env variable
CORS misconfiguration	Set ALLOWED_ORIGINS env variable to frontend domain only; enforce in FastAPI CORS middleware
JWT secret exposure	Add .env to .gitignore; rotate key before production deployment
File type validation	Add python-magic MIME sniffing on upload; allowlist accepted types per endpoint
Stale JWT state	Implement token refresh endpoint or automatic re-login on 401 response in SPA Axios interceptor

8. Testing Plan (Design Level)

8.1 Unit Testing Targets

Unit tests target individual functions and route handlers in isolation, using mocked database sessions and dependency overrides.

Module	Target Functions	Framework
Auth	hash_password, verify_password, create_access_token, decode_token	pytest
Course	generate_passcode (uniqueness, length=10), enrol_student (duplicate detection)	pytest
Assignment	validate_due_date (past date rejection), type_enum_check	pytest
Finance	create_finance_event (missing fields), verify_payment (status transition)	pytest
Admission	form_given_on auto-assignment, document path storage	pytest
Exam	exam_type_enum validation, filter logic (batch/type/room)	pytest
Equipment	borrow_quantity (exceeds available), return_update (quantity increment)	pytest

8.2 Integration Testing Targets

Integration tests exercise full request/response cycles through FastAPI with a test database.

Flow	Test Scenarios
Register → Login → Protected route	Valid credentials return JWT; invalid credentials return 401; student token rejected on admin endpoint (403).
Course create → Enrol → My courses	Created course appears in teacher list; student enrolls and appears in <code>my-courses</code> ; invalid passcode returns 404.
Assignment post → Submit → Grade	Student submission created; teacher retrieves submission list; marks and checked flag updated correctly.
Finance event → Payment submit → Verify	Payment created with <code>status = pending</code> ; admin verify endpoint sets <code>status = paid</code> with timestamp.
Admission multi-step submit → Admin list	Admission record created with non-null <code>form.given_on</code> ; admin list returns records sorted newest first.
Meeting schedule → RSVP → Archive	Meeting appears on public list; RSVP upsert works correctly; archived meeting absent from public list.

8.3 End-to-End (E2E) Testing Targets

E2E tests simulate real browser interactions against the fully running stack.

- **Student journey:** Register → Login → Enrol in course → View assignment → Submit file → View resource hub → Submit payment → Logout.
- **Teacher journey:** Login → Create course → Post assignment → View submission list → Grade submission → Archive course.
- **Admin journey:** Login → Create notice → Create exam → Create finance event → Verify payment → Review admission application.
- **Visitor journey:** Browse home page → View people directory → Read notice board → Complete multi-step admission form → Submit application.
- **Route guard:** Unauthenticated user attempts to access `/admin-dashboard` → Redirected to `/login`.

Recommended E2E framework: Playwright (cross-browser) or Cypress.

8.4 Known Hard-to-Test Areas

Area	Challenge and Approach
File upload handling	Multipart form submissions with real binary files are complex to mock. Use <code>pytest UploadFile</code> fixtures with small test PDFs and assert path storage rather than file content.
Mock payment gateway	The <code>MockPaymentGateway</code> component is a client-side simulation. Testing requires a Cypress/Playwright test that interacts with the component UI and captures the generated transaction ID.
JWT expiry	Testing token expiry requires either freezing time (use <code>freezegun</code> in Python tests) or issuing tokens with a 1-second TTL in test fixtures.
Batch-scoped filtering	Requires test data with multiple batches to verify correct isolation; must set up fixtures covering at least two distinct batch values.
Static file serving	<code>GET /resources/{filename}</code> requires a file to exist on disk; tests must create temp files in the resources directory and clean up after.
SQLite concurrency	Cannot be stress-tested under concurrent load with SQLite; performance/concurrency tests should use PostgreSQL in CI.

9. Exit Checklist

#	Checklist Item	Status
1	System overview is complete (problem, users, goals, out of scope documented in Chapter 2)	✓
2	Architecture and all components are documented with inputs, outputs, and design decisions (Chapter 3)	✓
3	Data model with all 21 entities, key relationships, and constraints is documented (Chapter 4)	✓
4	At least 2 critical flows are fully documented with trigger, steps, success outcome, and edge cases (Chapter 5 – 3 flows provided)	✓
5	Security and access model is defined: authentication approach, role-based authorisation, sensitive data handling, and audit plan (Chapter 6)	✓
6	Sprint implementation plan with completed features, pending features, technical risks, and mitigation plan is included (Chapter 7)	✓
7	Testing plan covers unit, integration, E2E targets and hard-to-test areas (Chapter 8)	✓
8	All requirements from SRS v2.0 (FR-AUTH through FR-ADMIN) are traceable to design components	✓
9	Team and project information including repository URL, commit tag, and related SRS version is included (Chapter 1)	✓
10	Revision history is present	✓

A. Requirements Traceability Matrix

Table A.1: SRS to SDD Traceability

SRS Req. ID	Backend Component	Frontend Component	SDD Section
FR-AUTH-01-05	userApi.py, auth.py	Login.jsx, SignUp.jsx, RequireAuth	§3.2, §6
FR-PUBLIC-01-06	Static JSON / public endpoints	HomePage.jsx, PeopleDirectory.jsx	§3.2
FR-NOTICE-01-04	AdminNoticeApi.py, StudentNoticeApi.py	AdminNotice.jsx	§3.2
FR-EVENT-01-04	AdminEventApi.py	AdminEvent.jsx, StudentEventShow.jsx	§3.2
FR-EXAM-01-04	AdminExamApi.py, StudentExamApi.py	AdminExam.jsx	§3.2
FR-ADMIT-01-05	AdminAdmissionHubApi.py, GuestAdmissionHubApi.py	AdmissionHub.jsx, ApplicationForm.jsx	§3.2, §5.3
FR-COURSE-01-05	courseApi.py, StudentMyClassesApi.py	MyCourses.jsx, EnrollCourse.jsx	§3.2, §5.1
FR-ASSIGN-01-04	StudentAssignmentApi.py, TeacherMyClassesApi.py	TeacherClassroom.jsx	§3.2, §5.1
FR-RES-01-02	main.py (static mount)	ResourceHub.jsx	§3.2, §4.4
FR-MEET-01-04	meetingApi.py	DepartmentalMeetings.jsx	§3.2
FR-EQUIP-01-04	AdminEquipmentApi.py, StudentEquipmentApi.py	AdminEquipment.jsx	§3.2
FR-FIN-01-06	financeApi.py, AdminFinance.py, StudentFinanceApi.py	AdminFinance.jsx, Finance.jsx	§3.2, §5.2
FR-TEACHER-01-04	teacherProfileApi.py, teacherDashboardApi.py	TeacherDashboard.jsx	§3.2
FR-STUDENT-01-04	StudentDashboardApi.py, StudentSettingsApi.py	StudentDashboard.jsx	§3.2
FR-ADMIN-01-02	All admin APIs	AdminDashboard.jsx	§3.2
NFR-SEC-01-06	auth.py (bcrypt, JWT)	RequireAuth, env vars	§6

(Continued)

SRS Req. ID	Backend Component	Frontend Component	SDD Section
NFR-SCALE-01-02	Stateless FastAPI, DB URL	env –	§3.3, §7.3

B. API Route Summary

Table B.1: Implemented API Route Summary

Prefix	Module	Description
/v1/auth	Auth	Register, login, password change
/v1/courses	Courses	Create, enrol, list, archive
/v1/teacher	Teacher Dashboard	Dashboard summary, activity logs
/v1/teacher/profile	Teacher Profile	Profile CRUD, research papers
/v1/meetings	Meetings	Create, list, archive, RSVP
/v1/finance	Finance	Fee events, student payments
/student/dashboard	Student Dashboard	Summary data
/student/notice	Notices	Student notice list
/student/event	Events	Student event list
/student/exam	Exams	Student exam list
/student/equipment	Equipment	Borrow, view orders
/student/assignment	Assignments	View, submit
/student/classes	My Classes	Enrolled course list
/student/settings	Settings	Profile update
/student/finance	Finance	View events, submit payment
/teacher/classes	Teacher Classes	Course management
/admin/notice	Admin Notices	Create, delete
/admin/events	Admin Events	CRUD events
/admin/exams	Admin Exams	CRUD exams
/admin/equipment	Admin Equipment	Inventory management
/admin/admission	Admin Admissions	List, delete forms
/guest/admission	Guest Admission	Submit application
/utility	Utility	Upload helpers
/resources	Static Mount	Serve uploaded files

C. Frontend Route Map

Table C.1: Frontend Route Map

Route	Component	Access
/	HomePage	Public
/login	Login	Public
/signup	SignUp	Public
/people	PeopleDirectory	Public
/chairman	AboutChairman	Public
/meetings	DepartmentalMeetings	Public
/notice-board	NoticeBoard	Public
/studentEventShow	StudentEventShow	Public
/admission-hub	AdmissionHub	Public
/apply	ApplicationForm	Public
/student-dashboard	Dashboard	Student
/my-courses	MyCourses	Student
/enroll-course	EnrollCourse	Student
/archived-courses	ArchivedCourses	Student
/resource-hub	ResourceHub	Student
/student-notice	StudentNotice	Student
/finance	Finance	Student
/settingspage	SettingsPage	Student
/teacher-dashboard	TeacherDashboard	Teacher
/teacher/classroom/:id	TeacherClassroom	Teacher
/assignments/:id/submissions	SubmissionList	Teacher
/admin-dashboard	AdminDashboard	Admin
/admin-finance	AdminFinance	Admin